Semaine 3

DOM avec styles CSS, variables globales et événements

Intro. à la programmation - Aut. 2022

Menu du jour 🍴

- Révision
- Variables globales, locales et constantes
- Booléens
  - Vrai ou faux ?
  - Opérateurs de comparaison
  - Opérateurs logiques
- Conditions
  - if
  - else
  - else if

Révision

- Semaine 1 :
  - Opérateurs mathématiques `+`, `-`, `*`, `/`, `()`
  - Opérateurs d'affectation `=`, `+=`, `-=`, `++`, `--`
  - Déclarer une variable : `let a = 3;`

Semaine 2 :

- Chaînes de caractères

```
>> let energie = "Je m'endors déjà";
```

- Concaténation avec +

```
>> "sa" + "lut"
<- "salut"
```

- Concaténation avec +=

```
>> let mot = "per";
   mot += "ruche";
<- "perruche"
```

Révision

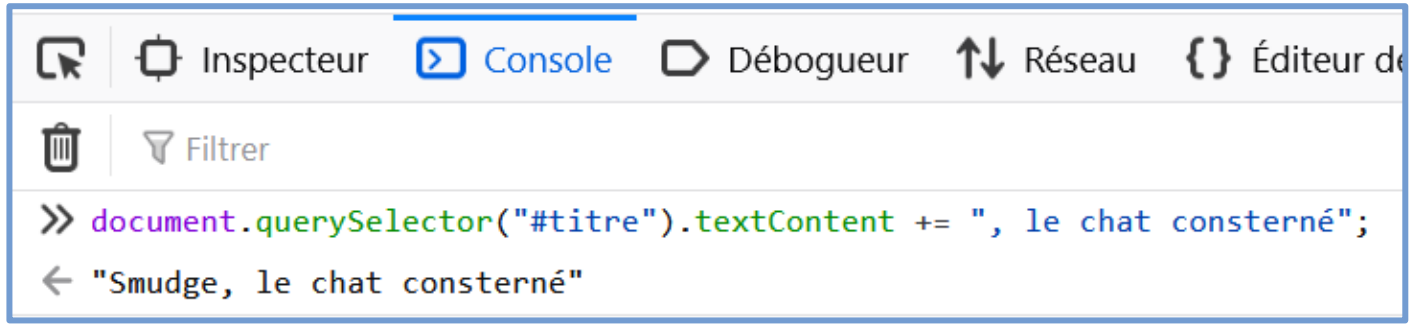
❖ Semaine 2 (suite)

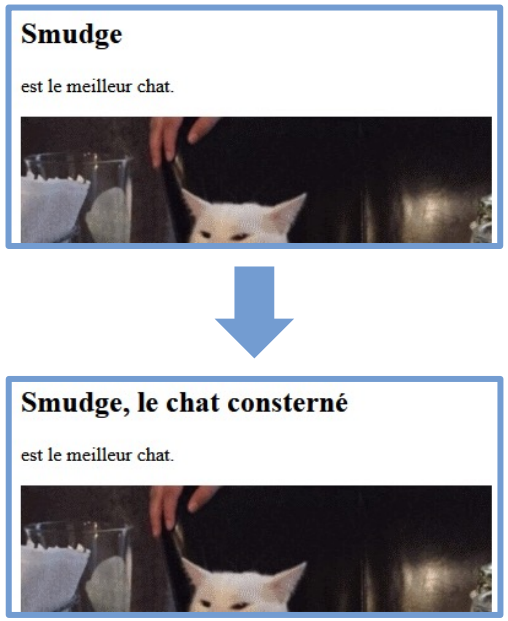
- ♦ document.querySelector() et .textContent

```
<div id="pikachu"> Pick a shoe </div>
```

```
document.querySelector("#pikachu").textContent = "Pikachu";
```

- ♦ .textContent et concaténation





Semaine 2 (suite)

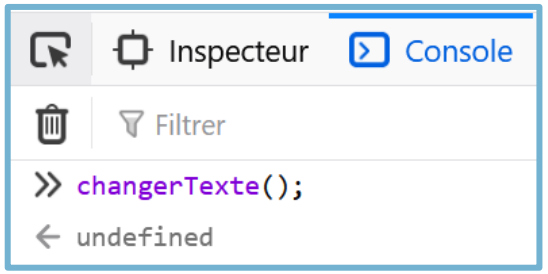
Les fonctions

Ce mot-clé sert à **déclarer** une fonction

Ceci est le **nom** de la fonction. C'est ce qui l'identifie.

Le morceau de code réutilisable est situé entre des **accolades** { ... }

```
function changerTexte(){
    document.querySelector("#titre").textContent = "Meme cat";
}
```



❖ Semaine 2 (suite)

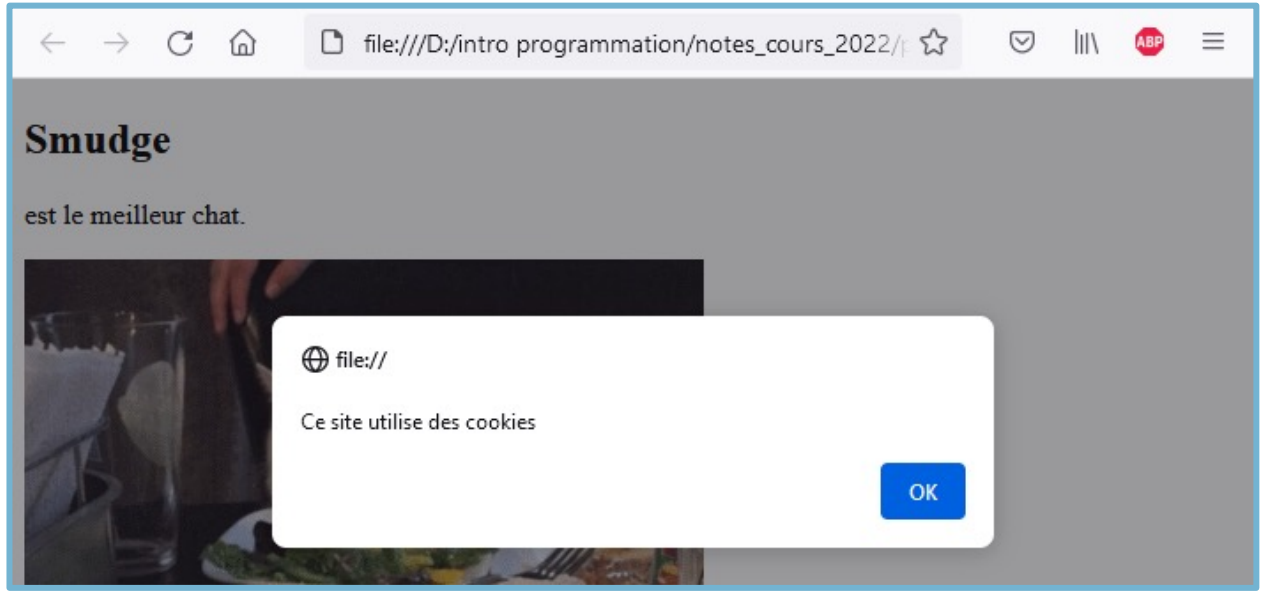
♦ Fonctions préexistantes

○ alert()

Inspecteur | Console | Débog

Filtrer

```
alert("Ce site utilise des cookies");
```



○ console.log()

Inspecteur | Console | D

Filtrer

```
let a = "Salut";
console.log(a + " à toi.");
```

```
Salut à toi.
```

❖ Constantes

- Nous avons vu comment déclarer des **variables** …

```
>> let nom = "Simone";
   let nombre = -3.5;
```

- Nous pouvons également déclarer des **variables constantes**
  - Il suffit de remplacer le mot-clé **let** par « **const** »

```
>> const PI = 3.1415;
   const USD_TO_CAD = 0.79;
```

Convention : Les variables constantes sont nommées en **MAJUSCULES** pour les reconnaître facilement !

  - Ces **variables constantes** peuvent être utilisée comme n'importe quelle autre

```
let circonference = PI * 1.65;
```

❖ Constantes

- **Particularité** : Leur valeur **ne** peut **pas** être **modifiée**. (Cela provoque une **erreur** dans le programme)

```
const PI = 3.1415;

PI = 3.14;
PI = PI + 1;
```

```
let PI = 3.1415;

PI = 3.14;
PI = PI + 1;
```

- Pourquoi les utiliser ?
  - Lorsqu'on sait qu'une **variable** n'est jamais censée changer de valeur, il est préférable d'en faire une **constante**. Comme ça, si jamais on change sa valeur par erreur, le programme nous avertit immédiatement de cette bévue.

Variables locales et globales

- Nous avons déjà vu comment déclarer une variable
  - Ex : `let a = 4;`
- Toutefois, l'**emplacement** dans le code (dans le fichier scripts.js, par exemple) où cette variable est **déclarée** est important.
  - La variable n'existe qu'à l'intérieur de la fonction où elle est déclarée. On ne peut pas l'utiliser ailleurs.

La variable texte est déclarée dans la fonction titre1()

```
function titre1(){
    let texte = "Natacha n'attache pas son chat";
    document.querySelector("#titre").textContent = texte;
}

function titre2(){
    document.querySelector("#soustitre").textContent = texte;
}
```

On peut utiliser texte ici sans problème.

On ne peut pas réutiliser la variable texte ici, puisqu'on n'est pas dans la fonction titre1(). Cela provoque une erreur.

❖ Variables **locales**

- Dans cette situation, texte est une variable locale. Elle ne peut être utilisée que « localement », c'est-à-dire seulement à l'intérieur de la **fonction** ou du « **bloc** » de code où elle est déclarée.

```
function titre1(){
    let texte = "Natacha n'attache pas son chat";
    document.querySelector("#titre").textContent = texte;
}

function titre2(){
    document.querySelector("#soustitre").textContent = texte;
}
```

🚫💩

Variables locales et globales

❖ Variables **globales**

- Une variable dite « globale » peut être utilisée n'importe où dans le code.

- Les variables globales doivent être déclarées en dehors de toute fonction, dans n'importe quel fichier JavaScript du projet Web.

- Ici, la variable gTexte est déclarée en dehors de toute fonction, au début du code.

- Elle est donc utilisable n'importe où dans les **fonctions** qui suivent.

```
let gTexte = "Ces seize chaises sont sèches";

function titre1(){
    document.querySelector("#titre").textContent = gTexte;
}

function titre2(){
    document.querySelector("#soustitre").textContent = gTexte;
}
```

Variables locales et globales

❖ Convention de nommage

- ♦ Pour dissiper le doute 🕵️, nous ajouterons toujours un g devant le nom d'une variable globale.
  - De plus, nous déclararons toujours les variables globales tout en haut du fichier .js

JS scripts.js × <> index.html

scripts.js > ...

Variables globales

```
let gScore = 15;
let gTexte = "Ces seize chaises sont sèches";
let gCouleur = "magenta";

function titre1(){
    let message = "Salut";
    document.querySelector("#titre").textContent = gTexte;
    console.log(alerte);
}
```

Variable locale

Variables locales et globales

❖ Constantes

♦ Globales ou locales

- Tout comme les variables déclarées avec « **let** », une variable **constante** peut être **globale** ou **locale**, selon l'endroit où on l'a déclarée.

```
const gUSD_TO_CAD = 0.79;

function calculerAireCercle(){
    const PI = 3.1415;
    let resultat = PI * PI * 3;
}
```

- **gUSD_TO_CAD** : Déclarée hors fonction, donc **globale**. Peut être utilisée n'importe où dans le code.
- **PI** : Déclarée dans une fonction, donc **locale**. Peut seulement être utilisée dans la fonction où elle a été déclarée.
  - À éviter : en général on veut que nos constantes soient globales. **!**

❖ Booléens

- C'est un autre **type de données**. (Nous avons vu **nombres**, **nombres décimaux** et **chaînes de caractères** pour le moment)
- Les *booléens* ont seulement 2 valeurs possibles
  - **true**
  - **false**
- Ils permettent d'exprimer que quelque chose est **vrai** ou **faux**. Exemples :
  - Avez-vous les yeux bleus ? 👁️
  - Êtes-vous majeurs ? 🍷🚬
  - Votre prénom contient-il la lettre T ? 🤔
  - Avez-vous déjà utilisé un extincteur ? 🔥

Par exemple, Simone, qui a les yeux verts, qui a 47 ans et qui n'a jamais assisté à une incendie, on pourrait répondre, dans l'ordre : false 👁️, true 🍷🚬, false 🤔, false 🔥.

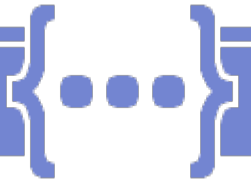
Booléens

- Ce sont des **valeurs** qu'on peut affecter à des **variables**
  - Attention ! Ce ne sont PAS des **chaînes de caractères** !

```
>> let a = true;
   let b = false;
← undefined
```

```
>> let c = "true";
← undefined
```

"true" est complètement différent de true. "true", c'est une chaîne de caractères quelconque.



❖ Opérateurs de comparaison

- Donnent un résultat qui est **true** ou **false**

Plus grand que	**>**	5.5 **>** 6.5 (**false**)
Plus grand ou égal	**>=**	5 + 2 **>=** 5 (**true**)
Plus petit	**<**	5 **<** 7 (**true**)
Plus petit ou égal	**<=**	5 **<=** 7 - 1 (**true**)
Égal	**==**	5 - 4 **==** 7 (**false**)
Pas égal	**!=**	5 **!=** 7 (**true**)

```
>> 1 < 2
← true
```

```
>> 6.5 >= 6.5
← true
```

```
>> let x = 5 < 3;
← undefined
>> x
← false
```

Au lieu d'assigner directement true ou false, on peut le faire via une vérification comme ceci.

❖ Opérateurs de comparaison (Avec des chaînes de caractère)

- Donnent un résultat qui est **true** ou **false**

- Égal **==** "allo" **==** "allo_" (**false**, différents)

- Pas égal **!=** "allo" **!=** "allo_" (**true**, différents)

- En ce qui concerne <, <=, > et >=, ils évaluent l'ordre alphabétique de deux chaînes de caractères, mais nous n'utiliserons pas ce genre de comparaisons.

❖ Priorité des opérateurs

- Ordre de priorité (Du premier au dernier)
  - **Parenthèses**
  - ***, /, %**
  - **+, -**
  - **<, <=, >, >=**
  - **==, !=**
  - **=**

```
» let b = 4 + 6 == 2 + 8;
```

```
» let a = 4 + 6 > 2 * 3 + 5;
```

Opérateurs de comparaison

❖ Booléens

- Nous connaissons donc maintenant les « données de type **booléen** » (**true** et **false**) et les **opérateurs de comparaison >, <, >=, <=, ==, !=**
  - Exemples

5 >= 5	**true**
5 < 5	**false**
"allo" != "allo "	**true**

❖ Opérateurs logiques

- Permettent de combiner plusieurs expressions de comparaison !

- ET **&&**
  - Les 2 conditions **doivent** être true

    `1 < 2 && 2 > 3` (**false**, car **2 > 3** n'est pas true)

- OU **||**
  - **Au moins une** condition doit être true

    `1 < 2 || 2 > 3` (**true**, car **1 < 2** est true)

- INVERSE **!**
  - Le booléen est inversé (true devient false, false devient true)

    `!(1 < 2)` (**false**, car **1 < 2** était true, mais on inverse)

|| représente deux barres verticales, et non deux L minuscules.

❖ Opérateurs logiques

- Exemples concrets

```
🦜 let prixPerruche = 5;
🎈 let prixBaudruche = 10;
🐝 let prixRuche = 15;
```

  - Le prix d'une **ruche** 🐝 est à la fois plus élevé que le prix d'une **perruche** 🦜 <u>**ET**</u> que le prix d'une **baudruche** 🎈

```
prixRuche > prixPerruche && prixRuche > prixBaudruche
```

```
» prixRuche > prixPerruche && prixRuche > prixBaudruche
```

Attention ! On <u>**ne doit pas**</u> écrire l'expression comme ceci :

🚫

```
» prixRuche > prixPerruche && prixBaudruche
```

❖ Opérateurs logiques

- Exemples concrets

🦜 `let prixPerruche = 5;`
🎈 `let prixBaudruche = 10;`
🐝 `let prixRuche = 15;`

  - **Au moins un** des trois prix est **plus élevé** que 13$

```
prixPerruche > 13 || prixBaudruche > 13 || prixRuche > 13
```

```
>> prixPerruche > 13 || prixBaudruche > 13 || prixRuche > 13
```

Attention ! On ne doit pas écrire l'expression comme ceci :

🚫 `prixPerruche || prixBaudruche || prixRuche > 13`

❖ Opérateurs logiques

- Exemples concrets

```
🕊️ let prixPerruche = 5;
🎈 let prixBaudruche = 10;
🐝 let prixRuche = 15;
```

  - Le prix d'une **perruche** 🕊️ **N'EST PAS** égal à 5

`!(prixPerruche == 5)` ou encore `prixPerruche != 5`

```
>> !( prixPerruche == 5 )
```

❖ Opérateurs logiques

- Exemples concrets

```
🦜 let prixPerruche = 5;
🎈 let prixBaudruche = 10;
🐝 let prixRuche = 15;
```

  - Le prix d'une **ruche** 🐝 **N'EST PAS** plus élevé que le prix de 2 **perruches** 🦜🦜.

```
!(prixRuche > 2 * prixPerruche)
```

```
>> ! ( prixRuche > 2 * prixPerruche)
```

❖ Bloc **if**

- Exécute un morceau de code seulement si une condition est respectée
- Syntaxe :

Entre les parenthèses **( )**, on retrouve la condition, qui DOIT être un booléen (true ou false)

```
if(/*...Condition...*/)
{
    // Code à exécuter si la condition est « true »
}
```

Entre les accolades **{ }** , on retrouve du code qui s'exécutera SEULEMENT si la condition est true.

Exemple dans la vie : Si tu as au moins 5$, tu peux faire un tour de montagne russe. Bien entendu, si tu détiens moins de 5$, il n'y aura tout simplement pas de tour de montagne russe. 😔

❖ Bloc **if**

- ♦ Exemples simplissimes

```
» if(true){
    console.log("Allo");
  }
```

Ici, la condition est true, alors nous allons exécuter le code dans le bloc.

```
» if(false){
    console.log("Allo");
  }
```

Ici, la condition est false, alors nous allons simplement sauter (skip) le bloc et passer à la suite sans exécuter son code.

- Bien entendu, on ne met jamais directement true ou false comme condition ! Sinon utiliser un bloc if ne sert à rien car on sait d'avance ce qui se produira.

❖ Bloc if

- ♦ Exemple plus pertinent
  - Ici, on écrit "majeur" dans l'élément avec l'id #personne seulement si la variable age est supérieure ou égale à 18.
  - Sinon, on saute le bloc de code du if.

```
» let age = 19;
  if(age >= 18)
  {
      document.querySelector("#statut").textContent = "Majeur(e)";
  }
```

```
» let age = 19;
  if(age >= 18)
  {
      document.querySelector("#statut").textContent = "Majeur(e)";
  }
```

❖ Conditions incohérentes

- 😬 ATTENTION ! Ne jamais confondre les opérateurs == et = !

```
>> if(x == 5){
     console.log("Valide");
   }
```

Ici, pas de problème ! On vérifie si x vaut 5. La condition est cohérente.

```
>> if(x = 5){
     console.log("Invalide");
   }
```

Problème : au lieu de poser une condition, on a mis « Je veux affecter 5 à la variable x ». Résultat ? C'est toujours considéré comme true.

❖ Bloc **else**

- Les blocs **if** **peuvent** être accompagnés d'un bloc **else**.
- Syntaxe :

```
if(/*...Condition...*/)
{
    // Code à exécuter si la condition est « true »
}
else
{
    // Code à exécuter si la condition est « false »
}
```

  - Le **else** n'est pas suivi d'une **condition**, car il est associé à la même condition que le **if**.
  - Le bloc de code sous le **else** s'exécute seulement si la condition est **false**.
    - Ainsi, il y a forcément **un seul des deux** blocs de code qui s'éxécutera.

Exemple dans la vie : Si tu m'aides à déménager, je te paye une pizza. Sinon, nous ne seront plus amis. Dans ce cas, le fait d'aider ET de ne pas aider ont une conséquence.

❖ Bloc else

♦ Exemples

- Ici, la condition du **if** est **true**. On va donc seulement exécuter le code du bloc **if**.
  - On met le texte "Enseignant(e)" à l'élément **#personne**.

```
let nom = "Maxime";
if(nom == "Maxime" || nom == "Maude" || nom == "Mathieu")
{
        document.querySelector("#personne").textContent = "Enseignant(e)";
}
else
{
        document.querySelector("#personne").textContent = "Étudiant(e)";
}
```

❖ Bloc else

♦ Exemples

- Ici, la condition du **if** est **false**. On va donc ignorer le code du bloc **if** et exécuter le bloc **else**.
  - On met le texte "Enseignant(e)" à l'élément **#personne**.

```
let nom = "Thérèse";
if(nom == "Maxime" || nom == "Maude" || nom == "Mathieu")
{
        document.querySelector("#personne").textContent = "Enseignant(e)";
}
else
{
        document.querySelector("#personne").textContent = "Étudiant(e)";
}
```

❖ Bloc **else if**

- Permet d'insérer **une ou plusieurs** conditions alternatives
- Syntaxe :

```
if(/*...Condition 1...*/)
{
    // Code à exécuter si la condition 1 est « true »
}
else if(/*...Condition 2...*/)
{
    // Code à exécuter si la condition 1 est « false » et la condition 2 est « true »
}
else
{
    // Code à exécuter si les conditions 1 et 2 sont « false »
}
```

Exemple dans la vie : Si tu es riche, je veux être ton meilleur ami. Sinon, si tu as une belle personnalité, je veux être ton ami. Sinon, je ne veux pas être ton ami.

❖ Bloc **else if**

♦ Exemple 1

- La première condition est **false**. On saute le bloc **if**.
- La deuxième condition est **true**. On exécute le bloc **else if** !
- On saute le **else** puisqu'un des blocs au-dessus était **true**.
  - On met le texte « Grand » à l'**élément**.

```
let a = 6;
if(a < 3)
{
 document.getElementById("element"). textContent = "Petit";
}
else if(a > 5)
{
 document.getElementById("element"). textContent = "Grand";
}
else
{
 document.getElementById("element"). textContent = "Moyen";
}
```

❖ Bloc **else if**

♦ Exemple 2

- La première condition est **false**. On saute le bloc **if**.
- La deuxième condition est **false**. On saute le bloc **else if**.
- On exécute le **else** puisque les deux blocs au-dessus était **false**.
  - On met le texte « Moyen » à l'**élément**.

```
let a = 4;
if(a < 3)
{
 document.getElementById("element"). textContent = "Petit";
}
else if(a > 5)
{
 document.getElementById("element"). textContent = "Grand";
}
else
{
 document.getElementById("element"). textContent = "Moyen";
}
```